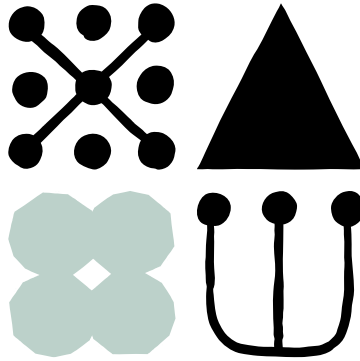


扩展 Managed Agents：将大脑与双手解耦

Scaling Managed Agents: Decoupling the brain from the hands

Anthropic · 2026-04-08 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



运行框架（harness）中包含着一些假设，而随着模型能力提升，这些假设会过时。Managed Agents 是我们用于长程 Agent 任务的托管服务，它围绕一组接口构建；即使运行框架不断变化，这些接口也能保持稳定。

请按照我们的文档，开始使用 Claude Managed Agents。

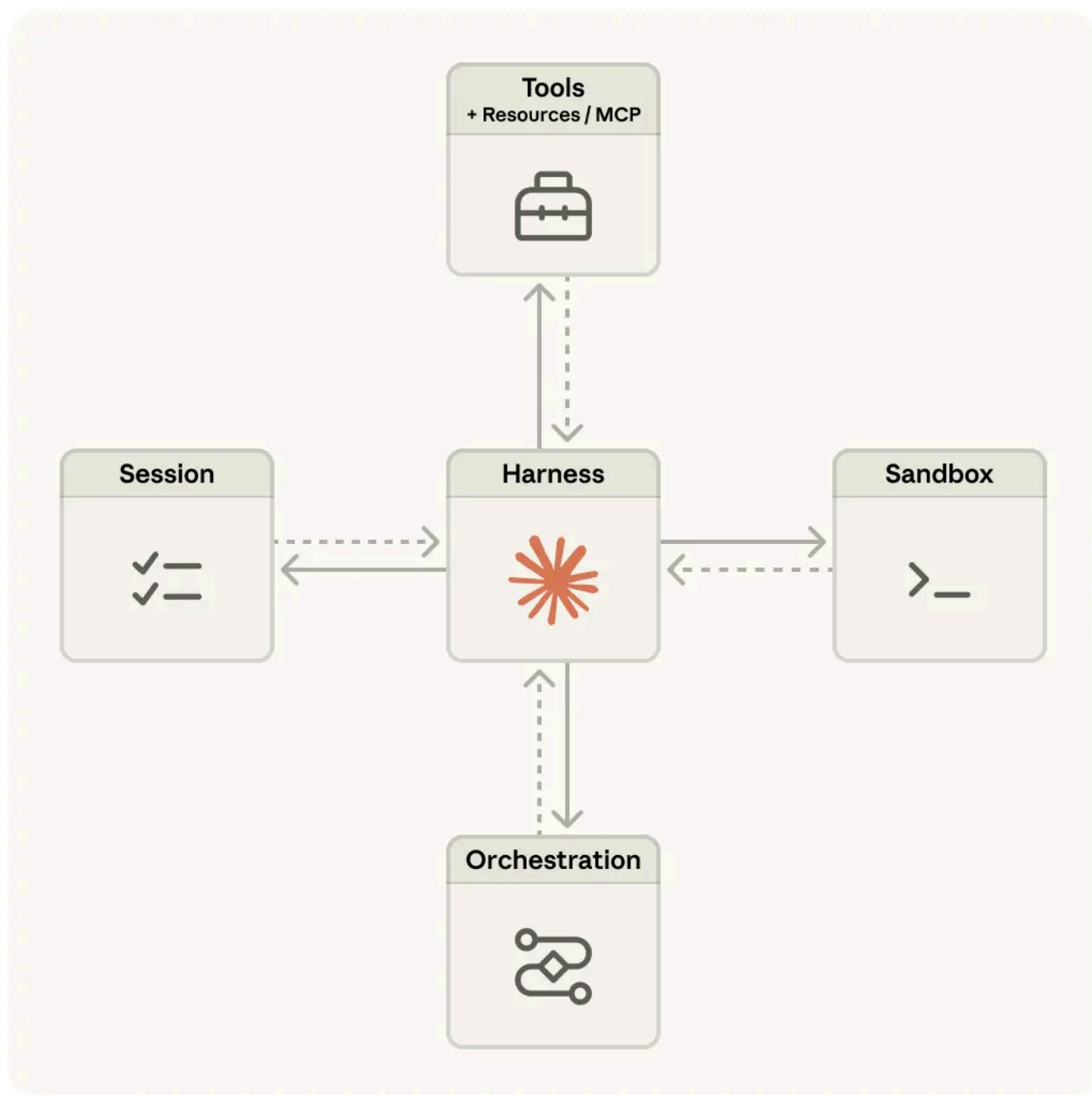
如何构建有效的 Agent，以及如何为长时间运行的任务设计运行框架，一直是本工程博客持续探讨的话题。这些工作有一个共同点：运行框架中包含着对 Claude 无法独立完成哪些事情的假设。然而，我们需要经常重新审视这些假设，因为随着模型能力提升，它们可能会过时。

举一个例子：在此前的工作中，我们发现，当 Claude Sonnet 4.5 察觉自己正在接近上下文上限时，会过早地结束任务——这种行为有时被称为“上下文焦虑”。我们通过在运行框架中加入上下文重置机制来解决这个问题。但当我们用同一运行框架用于 Claude Opus 4.5 时，却发现这种行为已经消失。重置机制成了多余的负担。

我们预计运行框架会继续演进。因此，我们构建了 Managed Agents：这是 Claude Platform 中的一项托管服务，通过一小组接口代你运行长程 Agent。我们希望这些接口能够比任何一种具体实现都更长久，包括我们今天正在运行的实现。

构建 Managed Agents 意味着要解决计算领域的一个老问题：如何为“尚未被构想出来的程序”设计系统。几十年前，操作系统通过将硬件虚拟化为足够通用的抽象——进程、文件——解决了这一问题，使之能够适用于当时尚不存在的程序。这些抽象的寿命比硬件更长。`read()` 命令不关心自己访问的是 20 世纪 70 年代的磁盘组，还是现代 SSD。上层抽象保持稳定，而底层实现可以自由变化。

Managed Agents 采用了同样的模式。我们将 Agent 的组成部分虚拟化为：会话（记录所有已发生事件、只能追加写入的日志）、运行框架（调用 Claude，并将 Claude 的工具调用路由到相应基础设施的循环），以及沙箱（Claude 可以在其中运行代码、编辑文件的执行环境）。这样一来，各部分的实现都可以替换，而不会影响其他部分。我们对这些接口应当具有什么形态有明确主张，但不限定它们背后运行什么。



不要养一只“宠物”

起初，我们把 Agent 的所有组件都放进同一个容器，这意味着会话、Agent 运行框架和沙箱共享同一个环境。这种做法有一些好处，例如文件编辑可以直接使用系统调用，也不需要设计服务之间的边界。

但把所有东西耦合在一个容器中，让我们遇到了一个老牌基础设施问题：我们养了一只“宠物”。在“宠物与牲畜”的类比中，宠物是有名字、需要精心照料、无法承受失去的个体，而牲畜则可以相互替换。在我们的情境中，服务器成了那只宠物：容器一旦故障，会话就丢失了；容器一旦无响应，我们就得悉心照料，让它恢复正常。

照料容器意味着要调试那些卡住、没有响应的会话。我们唯一的观察窗口是 WebSocket 事件流，但它无法告诉我们故障发生在哪里。结果，运行框架中的 bug、事件流中的丢包，或者容器离线，表现出来全都一样。为了查清问题，工程师必须在容器内部打开 shell。但这个容器通常还存有用户数据，因此，这种做法实际上意味着我们没有可用的调试能力。

第二个问题是，运行框架假设 Claude 处理的所有东西都与它放在同一个容器里。当客户要求我们将 Claude 连接到他们的虚拟私有云时，他们要么必须让自己的网络与我们的网络建立对等连接，要么必须在自己的环境中运行我们的运行框架。当我们想把运行框架连接到不同的基础设施时，固化在其中的一个假设就成了问题。

将“大脑”与“双手”解耦

我们最终采取的解决方案，是把我們所说的“大脑”（Claude 及其运行框架）与“双手”（执行动作的沙箱和工具），以及“会话”（会话事件的日志）解耦。每个部分都形成了一个接口，对其他部分只做很少的假设，并且都可以独立发生故障或被替换。

运行框架离开容器。将大脑与双手解耦，意味着运行框架不再位于容器内部。它像调用其他任何工具一样调用容器：`execute(name, input) → string`。容器变成了“牲畜”。如果容器宕机，运行框架就将故障捕获为工具调用错误，并反馈给 Claude。如果 Claude 决定重试，就可以按照标准流程重新初始化一个新容器：`provision({resources})`。我们不再需要悉心修复发生故障的容器，让它们恢复正常。

从运行框架故障中恢复。运行框架也变成了“牲畜”。由于会话日志位于运行框架之外，运行框架内没有任何东西必须在崩溃后保留下来。当一个运行框架发生故障时，可以通过 `wake(sessionId)` 启动一个新的运行框架，使用 `getSession(id)` 取回事件日志，并从最后一个事件继续执行。在 Agent 循环过程中，运行框架使用 `emitEvent(id, event)` 向会话写入事件，从而保留一份持久化的事件记录。

Component	Interface (pseudocode)	Satisfied by
Session	<pre>getSession(session_id) -> (Session, Event[]) getEvents(session_id) -> PendingEvent[] // not yet processed emitEvent(id, event)</pre>	Any append-only log that can be consumed in order from any event point and accepts idempotent appends — Postgres, SQLite, in-memory array, etc.
Orchestration	<pre>wake(session_id) -> void</pre>	Any scheduler that can call a function with an ID and retry on failure — a cron job, a queue consumer, a while-loop, etc.
Harness	<pre>yield Effect<T> -> EffectResult<T></pre>	Any loop that yields effects and appends progress to the Session.
Sandbox	<pre>provision({resources}) execute(name, input) -> String</pre>	Any executor that can be configured once and called many times as a tool— a local process, a remote container, etc.
Resources	<pre>[{source_ref, mount_path}]</pre>	Any durable store the container can fetch from by reference— Filestore, GCS, a git remote, S3.
Tools	<pre>{name, description, input_schema}</pre>	Any capability describable as a name and an input shape— MCP server, custom tool, etc.

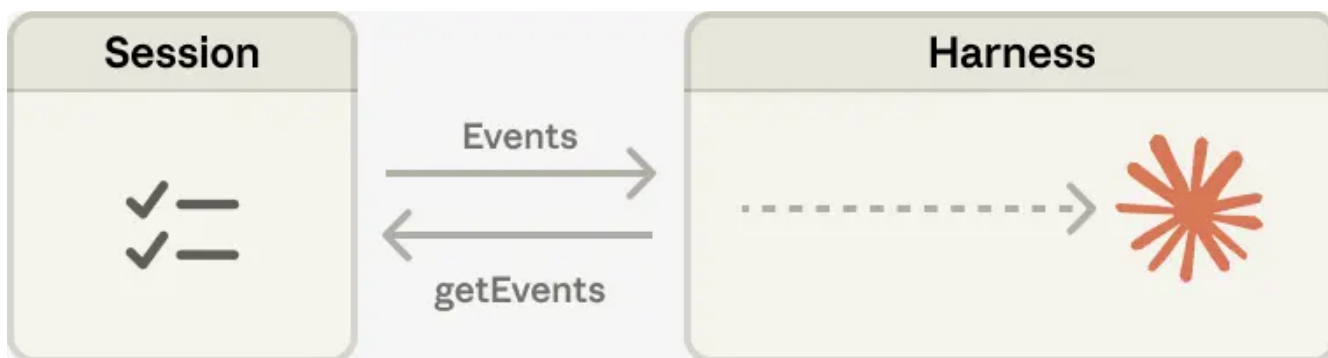
安全边界。在耦合的设计中，Claude 生成的任何不可信代码，都与凭证运行在同一个容器里——因此，提示注入只需要说服 Claude 读取它自己的环境即可。一旦攻击者拿到这些令牌，就可以创建新的、不受限制的会话，并将工作委派给它们。严格限制令牌的权限范围是一种显而易见的缓解措施，但这种做法隐含了一个假设：Claude 拿着受限令牌也做不了某些事情——而 Claude 正变得越来越聪明。结构性的解决办法是确保：Claude 生成的代码所运行的沙箱，永远无法访问这些令牌。

我们采用了两种模式来确保这一点。身份验证可以与资源绑定，也可以保存在沙箱之外的凭证保管库中。对于 Git，我们在沙箱初始化期间，使用各代码仓库的访问令牌克隆仓库，并将其接入本地 Git 远程配置。在沙箱内部可以执行 Git `push` 和 `pull`，而 Agent 始终无需直接处理令牌本身。对于自定义工具，我们支持 MCP，并将 OAuth 令牌存储在安全的凭证保管库中。Claude 通过专用代理调用 MCP 工具；该代理接收一个与会话关联的令牌，然后从保管库中获取对应的凭证，并调用外部服务。运行框架始终不会获知任何凭证。

会话并不等于 Claude 的上下文窗口

长程任务经常超出 Claude 上下文窗口的长度，而应对这一问题的常见方法，都涉及不可逆地决定保留哪些内容。我们在此前关于上下文工程的工作中探索过这些技术。例如，上下文压缩让 Claude 能够保存其上下文窗口的摘要，记忆工具则让 Claude 能把上下文写入文件，从而实现跨会话学习。这些方法还可以与上下文裁剪结合，选择性地移除旧工具结果或思考块等 token。

但选择性保留或丢弃上下文的不可逆决策可能导致失败。我们很难知道未来的轮次需要哪些 token。如果消息经过压缩步骤转换，运行框架会被压缩的消息从 Claude 的上下文窗口中移除；只有存储过这些消息，才能将其恢复。此前的工作探索过一种应对方式：将上下文存储为位于上下文窗口之外的对象。例如，上下文可以是 REPL 中的一个对象，LLM 通过编写代码，对它进行过滤或切片，从而以编程方式访问它。



在 Managed Agents 中，会话同样提供了这一好处：它充当位于 Claude 上下文窗口之外的上下文对象。但上下文并非存储在沙箱或 REPL 中，而是持久化存储在会话日志里。`getEvents()` 接口允许大脑按位置选取事件流的片段，从而查询上下文。这个接口可以灵活使用：大脑可以从上次停止阅读的位置继续，也可以回退到某个特定时刻之前的几个事件，了解事情的来龙去脉，或重新阅读某个特定动作之前的上下文。

获取到的任何事件，都可以先在运行框架中转换，再传入 Claude 的上下文窗口。这些转换可以是运行框架规定的任何操作，包括为了提高提示缓存命中率而组织上下文，以及其他上下文工程处理。我们将会话中可恢复的上下文存储，与运行框架中可自由实施的上下文管理分开，因为我们无法预测未来模型具体需要什么样的上下文工程。这些接口将上下文管理交由运行框架负责，只保证会话持久可靠，并且可供查询。

多个大脑，多双手

多个大脑。将大脑与双手解耦，解决了客户最早提出的一类问题。当团队希望 Claude 操作他们自己 VPC 中的资源时，唯一的办法是让我们的网络与他们的网络建立对等连接，因为承载运行框架的容器假设每项资源都近在身旁。当运行框架不再位于容器中时，这一假设就消失了。同样的改变还带来了性能收益。最初，我们把大脑放进容器，这意味着有多少个大脑，就需要多少个容器。对每个大脑而言，容器配置完成之前无法进行任何推理；每个会话都必须预先承担完整的容器启动开销。每个会话——即使根本不会使用沙箱——都必须克隆仓库、启动进程，并从我们的服务器获取待处理事件。

这段空等时间体现在首 token 延迟（time-to-first-token, TTFT）中，它衡量的是会话从接受任务到生成第一个响应 token 之间的等待时间。TTFT 是用户感受最明显的延迟。

将大脑与双手解耦，意味着只有在确实需要容器时，大脑才会通过工具调用 `(execute(name, input) → string)` 配置容器。因此，不需要立即使用容器的会话，就不必等待容器就绪。只要编排层从会话日志中拉取到待处理事件，就能开始推理。采用这一架构后，我们的 p50 TTFT 降低了约 60%，p95 则降低了超过 90%。扩展到多个大脑，只需要启动多个无状态的运行框架，并仅在需要时为它们连接双手。

多双手。我们还希望能够为每个大脑连接多双手。在实践中，这意味着 Claude 必须对多个执行环境进行推理，并决定将工作发到哪里——这比在单个 shell 中操作需要更强的认知能力。最初，我们让大脑待在单个容器里，是因为早期模型还不具备这种能力。随着智能水平提升，单个容器反而成了限制：一旦这个容器发生故障，大脑所连接的所有“手”的状态都会丢失。

将大脑与双手解耦，使每一只手都成为一个工具：`execute(name, input) → string`。传入名称和输入，返回一个字符串。这一接口支持任意自定义工具、任意 MCP 服务器，以及我们自己的工具。运行框架不知道沙箱究竟是一个容器、一部手机，还是一个宝可梦模拟器。而且，由于任何一只手都不与某个大脑耦合，大脑之间可以相互移交这些手。



结语

我们面对的是一个老问题：如何为“尚未被构想出来的程序”设计系统。操作系统之所以能延续数十年，是因为它们把硬件虚拟化成了足够通用的抽象，能够适用于当时尚不存在的程序。通过 Managed Agents，我们希望设计一个系统，容纳未来围绕 Claude 构建的运行框架、沙箱及其他组件。

Managed Agents 是遵循同样理念的元运行框架（meta-harness），它不预设 Claude 未来需要哪一种具体的运行框架。相反，它是一套具有通用接口、允许多种不同运行框架存在的系统。例如，Claude Code 是一个出色的运行框架，我们广泛将其用于各种任务。我们也展示过，针对特定任务设计的 Agent 运行框架，可以在狭窄领域表现优异。Managed Agents 可以容纳其中任何一种，随着时间推移适应 Claude 的智能水平。

设计元运行框架，意味着对 Claude 周边接口的形态持有明确主张：我们预计 Claude 需要操作状态（会话）和执行计算（沙箱）的能力。我们也预计 Claude 需要扩展到多个大脑和多双手的能力。我们设计这些接口，是为了让它们能够在长时间跨度内可靠、安全地运行。但对于 Claude 将需要多少个大脑、多少双手，以及它们位于何处，我们不做任何假设。

致谢

作者：Lance Martin、Gabe Cernaj 和 Michael Cohen。感谢 Nodir Turakulov 和 Jeremy Fox 就这些话题展开富有帮助的交流。特别感谢 Agents API 团队以及 Jake Eaton 的贡献。